

CS460 Project Assignment

VERSUS: A tournament bracket prediction and voting platform

Deadlines

Database design report: Friday, June 12, 2026, at 11:59PM.

Final report and implementation: Thursday, June 25, 2026, at 11:59PM.

Skeleton code with the project structure and the database connection helper is included alongside this report. Follow the README to install MySQL, create the database, and run the Flask server locally.

Purpose of the project

In this project, you will design, implement and document a database system for a tournament bracket prediction and voting application called VERSUS. You will also provide the web interface to the database. Users create single-elimination brackets of 4, 8, 16, or 32 entrants, submit their predictions for the winners of each matchup, vote during each round, and eventually crown a champion. The final system should be functional and will look similar to a stripped-down ESPN bracket challenge. The project is to be done individually.

Data

The system should manage the following information:

Users

Each user is identified by a unique user id and has the following attributes: username, email, password, optional bio, and the date they joined. The username and email must be unique. A user can host brackets, submit predictions, vote, follow other users, and leave comments on matchups.

Brackets

Each bracket is identified by a unique bracket id and has the following attributes: host (a user), title, description, entrant count, status, an optional prediction deadline, and the date of creation. The entrant count must be 4, 8, 16, or 32. The status indicates where the bracket is in its lifecycle, with possible values of draft, predictions_open, round_1 through round_5, or completed. Each bracket contains exactly entrant_count entrants and exactly (entrant_count - 1) matchups arranged across $\log_2(\text{entrant_count})$ rounds.

Entrants

Each entrant is identified by a unique entrant id and must belong to a bracket. Each entrant has a seed number, a name, and an optional image URL. The seed number is unique within its bracket. Seed 1 plays seed 2 in Round 1, seed 3 plays seed 4, and so on.

Matchups

Each matchup is identified by a unique matchup id and must belong to a bracket. Each matchup has a round number, a slot number, two entrant slots (A and B), a winner, and running vote counters. The combination of bracket, round, and slot must be unique. The winner is NULL until a round is closed. Round 1 matchups are created with both entrants set, while later-round matchups are created with NULL entrants and are filled in as previous rounds close.

Predictions

Each prediction is identified by a unique prediction id and is associated with a user and a matchup. A prediction records which entrant the user picked, whether the pick turned out to be correct, how many points it earned, and the date it was submitted. A user can have at most one prediction per matchup. Predictions may only be submitted while the bracket is in the predictions_open state. This rule is enforced inside the database by a BEFORE INSERT trigger.

Votes

Each vote is identified by a unique vote id and is associated with a user, a matchup, and the entrant the user voted for. A user can vote at most once per matchup. Votes may only be cast on a matchup whose round equals the bracket's current round. This rule is also enforced by a BEFORE INSERT trigger.

Achievements

Each achievement is identified by a unique code, such as 'bracket_maker' or 'locked_in', and has a name and description. The user_achievements table records which user earned which achievement and when. Achievements are awarded automatically by AFTER INSERT triggers. Hosting your first bracket awards the bracket_maker achievement, and submitting your 10th prediction awards the locked_in achievement.

Follows

Each follow record links two distinct users (a follower and a followed user) with a timestamp. The pair (follower_id, followed_id) is the primary key. A CHECK constraint prevents a user from following themselves.

Comments

Each comment is identified by a unique comment id and is associated with a user and a matchup. A comment has a text body and a creation timestamp. Comments appear inline on the matchup card.

Use cases

The following interactions with the system should be implemented.

User management

Becoming a registered user. Before hosting brackets, submitting predictions, or voting, a user must register by providing a username, email, and password. If a user already exists with the same username or email, an error message should be produced. Passwords are stored hashed, not in plain text. Login verifies the password against the stored hash and sets a signed session cookie that holds only the user id.

Following other users. A logged-in user can follow or unfollow any other user. Self-follows are rejected by a database-level CHECK constraint. A user's profile page lists their followers and the users they follow.

Profile pages. Each user has a profile page that displays their bio, total prediction points across all brackets, number of predictions made, number of correct picks, achievements earned, brackets they have hosted, and their follower and following lists.

Bracket and matchup management

Bracket creation. After registration, users can host a new bracket by providing a title, description, entrant count (4, 8, 16, or 32), and the names of all entrants in seed order. The bracket row, all entrant rows, and all matchup shells for every round are inserted in a single atomic transaction. If any insert fails, none of them are persisted. As an example, an 8-entrant bracket creates exactly 7 matchups: 4 in Round 1 (with both entrants set), 2 in Round 2, and 1 Final. Later-round matchups are created with NULL entrant slots.

Bracket browsing. Every visitor to the site, registered or not, should be allowed to browse the list of brackets and view individual brackets. The bracket's status determines which interactive controls are shown. In the `predictions_open` state a prediction form is shown, in the `round_N` state vote buttons are shown on Round N matchups, and in the `completed` state a champion banner with a link to the champion path is shown.

Predictions. While a bracket is in the `predictions_open` state, any logged-in user may submit one prediction per Round 1 matchup. All of a user's picks for a given submission are written in a single transaction and share the same `submitted_at` timestamp. A UNIQUE constraint on (`user_id`, `matchup_id`) blocks duplicate picks.

Voting. Once the host advances the bracket to `round_1`, and again for each subsequent round, logged-in users may vote on the matchups in the current round. Each vote is recorded as an INSERT into `votes` followed by an atomic UPDATE that increments the matchup's vote counter (`votes_a` or `votes_b`) by one. The naive read-then-update pattern is not allowed because concurrent votes would be lost.

Closing a round. The host of a bracket can close the current round, which calls a stored procedure `close_round(bracket_id, round)` that performs four tasks in one transaction. First, it sets the winner of each matchup in the round based on the vote totals. Second, it scores every prediction made for that round. Third, it promotes the winners into the entrant slots of the next round's matchups. Fourth, it advances the bracket's status to the next round, or marks the bracket as completed if the final round was just closed.

Statistics and discovery

Leaderboard. A global leaderboard ranks all users by their total prediction points across all brackets. It is generated by a single SQL query that uses the `RANK()`, `DENSE_RANK()`, and `PERCENT_RANK()` window functions. Tied users share the same rank. The current user's row is highlighted on the page.

Champion path. For a completed bracket, a champion-path page lists the winning matchups of the eventual champion from Round 1 to the Final. The list is produced by a single recursive CTE that walks the bracket tree backwards along the path of the champion.

Achievements. Two achievements are awarded automatically by AFTER INSERT triggers in the database. The bracket_maker achievement is awarded when a user hosts their first bracket. The locked_in achievement is awarded when a user submits their 10th total prediction. A user's earned achievements are displayed on their profile and shown as small chips in the nav bar.

Social interaction

Comments. Logged-in users may leave a comment on any Round 1 matchup of any bracket. Comments are visible to all visitors and appear inline below the matchup card. For the scope of this project, comments cannot be edited or deleted once posted.

Administration

SQL console. A user whose username is 'admin' should have access to a page that runs arbitrary SQL against the live database. This is a demonstration tool used to show EXPLAIN plans and to verify queries during grading. All non-admin users should be denied access.

Rules of the Project

Implementation Tools

We will provide you with all the tools you need to complete this project. All queries to the database will be made via SQL statements. For this project, you are expected to be familiar with Flask, Python, and SQL. Everything else will be explained or automated for you. The project is to be done individually. You are allowed to talk with each other about the project but you are not allowed to share code or queries. If you have questions about the project you should ask the Instructor first.

Overview of Project Phases

The project is divided into 2 phases. The first phase covers the database design and the second phase covers the API implementation and the website. Everything is submitted together at the end after both phases are complete.

Phase I - Database Design

Initially, you must design and create the database for the application. This includes creating the tables that store the data, and designing rules for how these tables relate to each other so that the data can be combined in meaningful ways. As part of the report for this phase, you will turn in an E-R diagram of your database design and the translation to the relational schema (CREATE TABLE statements). You must also provide all the constraints and the SQL code used to enforce them (CHECK or ASSERTION clauses).

Phase II - API Implementation and Web Site

In this phase, you should write the SQL queries and some Python code that implements an interface to your database and executes the queries. This part will be used in the next phase for the web interface. Finally, you will create a web site for your database and add the bracket prediction and voting functionality.

Deliverables

You should provide two deliverables for this project. The first one is the database design and the second is the final system.

Here are more details:

- Friday, June 12, 2026 at 11:59PM: You need to provide a report with the E-R diagram, the relational schema, the assumptions that you make, and the integrity constraints. Submit the report by Gradescope.
- Thursday, June 25, 2026 at 11:59PM: The final report with the code, and the additional assumptions you made along with the limitations of your system. In particular, you should submit the following by Gradescope: (1) the final report containing the final schema and any additional assumptions and constraints you made, and (2) a zip file with the versus directory (all your code), or alternatively the versus directory itself.